

Selection

Revise 360 lesson plan — 60 minutes

Unit	Beyond OCR J277: a Python pathway supporting 2.1, 2.2 and 2.3
Lesson	Selection (lesson 3 of the unit)
Lesson ID	pr-103
Experience ID	pr-103
Plan ID	PLAN-PR-L03
Content version	fd2549e1dbc2
Website	https://revise360.co.uk/experience.html?id=pr-103

Big question, outcomes and success criteria

Big question — Why does the order the branches are written in change what an if ... elif ... else actually does?

Prerequisites

The lesson before this one is pr-102. Pupils should already have finished it: the starter below retrieves from it.

Earlier in the unit: pr-101.

Assessable outcomes

1. **pr-103-o1** — know how to write a condition using the comparison operators
2. **pr-103-o2** — understand how if, elif and else choose between several outcomes
3. **pr-103-o3** — know how to join two conditions together with and, or and not

Success criteria the pupil can check against

Taken from the briefs the independent activities themselves show, so these are the words the pupil sees. Their work has met the outcome when it does these things.

- The user types in one whole number. **pr-103-s1-a6**
- One line of output, which is True or False. **pr-103-s1-a6**
- The user types in two lines, one word on each. **pr-103-s1-a7**
- Three lines of output, each one True or False. **pr-103-s1-a9**
- Exactly one of the three lines is True. **pr-103-s1-a9**
- Display exactly one word: pass or fail **pr-103-s2-a5**
- The user types in one word. **pr-103-s2-a6**
- The word `"amber"` is written into your program. **pr-103-s2-a6**

Vocabulary, resources and preparation

Key vocabulary — Selection, Condition, elif, Nested selection

Resources

SLIDES	presentations/PR_L03_Selection_Lesson.pptx
WORKSHEET	worksheets/PR_L03_Selection_Worksheet.docx
ONLINE EXPERIENCE	experience.html?id=pr-103

Before the lesson

1. Sign in on the **teacher dashboard** (`teacher.html`) and open **Class logins**. Enter the class or group and the usernames, one per line, then print the login cards. A pupil cannot sign in without one, so do this before the lesson rather than in it.
2. Print the worksheet, one per pupil.
3. Open the slides and the experience once yourself, on the machine you will be teaching from, so the Python runtime and the 360 image are already cached.
4. Check headsets or devices are charged and on the school network.

How pupils get in

1. Go to `topics.html` on `revise360.co.uk`.
2. Sign in with the **username and PIN from the login card**. The class is already set against the card.
3. Choose the topic, then this lesson's card.

Worth knowing before you promise anything. In this build `APP_CONFIG.backendUrl` is empty, so logins, login cards and progress are stored on the device the pupil used. The teacher dashboard says so too. A pupil who moves to a different machine starts again, so keep a class on the same devices within a unit, and treat the paper worksheet as the record that travels.

Teaching sequence

Sixty minutes including transitions. The online block is the one to move if your lesson is shorter: see the adaptation at the end.

MIN	PHASE	TEACHER	PUPILS	EVIDENCE TO CHECK
0-5	Retrieve	Starter on the board or the worksheet, from the last lesson and this lesson's prerequisites.	Answer the starter on the worksheet.	The starter box on <code>PR_L03_Selection_Worksheet.docx</code> is filled in.
5-8	Review the starter	Take answers; correct the one most got wrong before moving on.	Correct their own starter.	A correction written, not just a tick.
8-16	Explain and model	The new idea, then the worked example, typed live rather than shown finished.	Watch the example being built, and say what each line does.	They can name the line that does the job, not just copy it.
16-19	Set up and settle	Sign-in, launch, and what to record on paper before anyone types.	Sign in, open the lesson, read the task before typing.	Everyone is at station 1 activity 1, not typing yet.
19-48	Online: work the stations in order	Pupils work; you circulate and read the editor over shoulders.	Work the stations in order, running and checking as they go.	The station badges fill in; the editor shows their own program, not the starter.
48-53	Check understanding	The hinge question, hands down, then the reteach if the class splits.	Answer the hinge question, hands down.	The split across the options, before any reteach.
53-58	Apply or extend	The challenge activity, or the exam practice on the worksheet.	The Challenge activity, or the exam practice.	worksheet, Exam practice a) to c)
58-60	Exit and set up the next lesson	The exit question and what to finish before next time.	Write the exit answer.	One sentence that answers the big question.

The explanation and the worked example

This is what the walls of the experience say, station by station. Teach the first two or three from the front before anyone puts a headset on; the rest the pupils read for themselves.

1. Comparing values

- A comparison gives a boolean back: either True or False.
- The comparison operators are ==, !=, >, <, >= and <=.
- Common mistake: comparing with one =, when one = stores a value and two compare them.

Challenge on this wall: What does `a > b` print when `a` is 5 and `b` is 5?

2. if and else

- `if` runs the indented block below it only when its condition is True.
- `else` catches every other case, and needs no condition of its own.
- Common mistake: missing the colon, or not indenting the block underneath it.

Challenge on this wall: Why does a rule worded as '20 or more' need `>=` rather than `>`?

3. elif

- `elif` adds another condition, which is only tested if the ones above it were False.
- Python stops at the first branch that is True and skips everything below.
- Common mistake: putting a wide band first, so it catches everything and the later branches never run.

Challenge on this wall: Why must the bands of a grade boundary list be tested in order?

4. AND, OR and NOT

- `and` is True only when both sides are, and `or` is True when at least one side is.
- `not` flips a condition over, so `not found` is True whenever `found` is False.
- Common mistake: writing `n == 3 or 7`, which Python reads as something quite different.

Challenge on this wall: How would you test in one condition whether a number is either 3 or 7?

5. Decisions inside decisions

- A decision can sit inside another, and the inner one is only reached if the outer was True.
- Each level of nesting is indented one more step in than the one above.
- Common mistake: nesting where a single `and` would have said the same thing far more clearly.

Challenge on this wall: When is a nested `if` clearer than one condition joined with `and`?

6. Putting it together

- Work the boundaries out first, then write the bands in the order the rules apply.
- Test on the edge of every band, because that is exactly where conditions go wrong.
- Common mistake: using `>` where the rule says 'or more', which loses the boundary value.

Challenge on this wall: Which three values would you test on a program whose bands change at 30 and 50?

The example to model

A comparison asks a question about two values. The answer is always one of two words: True or False. Python works the comparison out and hands that word back, so `print()` can display it.

```
print(7 > 3)
print(7 < 3)
```

It displays **True / False**

Decisions to model out loud, not just the finished answer: say why you chose that name, why that line comes before the next one, and what you would try first if it did not work. The example deliberately uses different data from the task, so a pupil cannot copy it and be finished.

Retrieval and the check for understanding

Starter — A program has three branches: if $n > 20$ it prints big, elif $n > 10$ it prints middle, and else it prints small. Write down what is printed when n is 30, when it is 11, and when it is 10.

Answer space: 3 lines on the worksheet.

The answer is in the teacher answers for this unit, not on this plan: a plan left on a desk should not give it away.

Hinge question

Use this one, hands down, before anyone moves on to independent work. It is a real activity from the experience (pr-103-s4-c2), so the wording a pupil sees matches the wording you say.

What will this program display?

Correct: True False

What a pupil might choose instead

- False True
- True True

Count the hands on each. If more than a few of the class are on one wrong option, that option is the misconception to teach against before anyone goes on: go back to the station this question sits on and work through its example again, aloud, before the independent work.

The course's own response to a wrong answer here: or needs only one True side. On the second line neither side is True, so the whole thing is False.

One response, not four. one response covers every wrong option; it does not say what to revisit for each one, so decide your own reteach for each option before the lesson - the list above is the start of it.

Misconceptions this lesson addresses

- comparing with one =, when one = stores a value and two compare them. [wall panel](#)
- Read the error. input() always hands back text, and text cannot be compared with a number. [pr-103-s1-a8, a Fix it activity](#)
- missing the colon, or not indenting the block underneath it. [wall panel](#)
- Look at the end of the if line and compare it with the end of the else line. [pr-103-s2-a7, a Fix it activity](#)
- Which of the two displayed lines should happen whatever the number is? [pr-103-s2-a8, a Fix it activity](#)
- putting a wide band first, so it catches everything and the later branches never run. [wall panel](#)
- The second question is being asked even when the first was already True. [pr-103-s3-a7, a Fix it activity](#)
- writing $n == 3$ or 7, which Python reads as something quite different. [wall panel](#)

Guided to independent practice

STAGE	ACTIVITIES	WHAT IT IS
Guided	27	Support is in the activity: an example to run, an output to predict, one thing to change, a gap to fill.
Independent	22	The pupil writes or fixes it with the brief and nothing else.
Challenge	6	The hardest activity on each station. It is not optional and it is not extra: it is the last step of the station.

Scaffolds already in the product, and how they fade

- Hint** is a ladder, not a door: the first rung names the idea, the next shows the shape of the code, the next how it starts, the last animates the technique on different data. A pupil asks for each rung. None of them is the answer.
- Run** is free and unmarked; **Check my answer** marks. A pupil may run as often as they like, which is what makes trial and error safe.
- I need help** says to ask you, and says plainly that the question still has to be completed. It does not notify anyone and it does not unlock anything.

- The worked example appears on every early activity and on none of the late ones. That fade is deliberate; it is not a missing example.

Accessibility

- **Read aloud** on every task window reads the task and its brief, with the prescribed values read as values rather than as punctuation.
- Text is 16px or larger throughout, and a question window scrolls rather than shrinking its type.
- Prescribed data is boxed and in a monospace face as well as coloured, so it still reads as code in black and white, in a high-contrast mode, and to a pupil who does not separate green from amber.
- **Lower literacy and EAL:** pre-teach the vocabulary above; let the pupil say the key fact before writing it; accept the key fact in two or three words.
- **SEND:** the sequence is compulsory but the pace is not. A pupil may finish a station next lesson; the progress saves after every answer and reopens at their earliest unfinished question.
- A headset is never required. Every experience works on a laptop or a tablet with a mouse or a finger.

Delivering the online experience

Before anyone enters

- Worksheet out, pen in hand. The rule is write first, then answer at the badge: the first answer is the one that counts.
- Say where the stations are. They are numbered around the room: 1 and 2 to the right, 3 and 4 behind, 5 and 6 to the left, and the final challenge on the floor.
- Say that a badge marked **Not yet** is not a fault. It opens when the station they are on is finished.

The stations, and what to record at each

#	STATION	ACTIVITIES	RECORD ON PAPER
1	Comparing values	9	A value that can only be True or False is named after George Boole, so it is called a...
2	if and else	9	Python uses indentation where most languages use brackets, so the layout is part of the...
3	elif	9	elif is short for two words that some languages make you write out in full:...
4	AND, OR and NOT	9	Python stops reading an and as soon as one side is False, which is called...
5	Decisions inside decisions	9	Deeply nested ifs are so hard to follow that programmers call the shape the...
6	Putting it together	9	A mistake at the edge of a band is so common that it has its own name: an...
final	Final challenge: true or false	1	the answer to the station's challenge

Where to pause

Stop the class after station 2 and take answers to its challenge out loud. That is the point where a pupil who has misread the technique can still be put right cheaply.

Afterwards

Headsets off and down. Ask three pupils for the key fact from a station of your choosing, in their own words and not read off the sheet. Then the exit question.

Python: the order is not negotiable. Each activity teaches what the next one needs, so a pupil works through them in order and completes every one. A shorter lesson, homework or a slow worker does not authorise skipping: the pupil stops where they are and resumes at their earliest unfinished question next time, because progress saves after every answer.

A paper activity, a help request or a tick on this plan does not mark an online question complete, and there is no teacher bypass. If a pupil is stuck after the hint, that is the moment to teach them, not to move them on.

Assessment, evidence and unfinished work

ID	QUESTION	MARKS	WHERE
pr-103-x1	A program should print cheap when a number is 2 or less and dear when it is more. A pupil writes <code>if n > 2</code> then prints cheap, and else prints dear. Identify the logic error and write the two lines corrected.	3	worksheet, Exam practice a); deck, Exam practice slide
pr-103-x2	Write a program that reads a whole number and prints small when it is 10 or less, medium when it is more than 10 but 100 or less, and large when it is more than 100.	4	worksheet, Exam practice b); deck, Exam practice slide
pr-103-x3	Write a program that reads a whole number and then a word, and prints open only when the number is 3 or 7 and the word is not locked. Otherwise it prints refused.	4	worksheet, Exam practice c); deck, Exam practice slide

Expected answers

Model answers and marking guidance are teacher-only and are not printed on this plan. They are in the unit's teacher guide pack, and the per-activity marking for this lesson is in `answers/records/pr-103.json`, which is not published with the website.

Every programming activity in this lesson is marked by running the pupil's program against its test cases, so the mark is the program's behaviour and not its appearance. The model solutions are in `answers/codebank/pr-103.json`.

Common errors

The wrong options the activities offer are the errors this lesson expects. The commonest to watch for:

- **What will this program display?** — False True; True True
- **What will this program display?** — True False; False False
- **What will this program display?** — inside outside; inside
- **What will this program display?** — merit; pass merit
- **What will this program display?** — False True; True True

Unfinished work

Record where each pupil stopped, not a tick. They resume at their earliest unfinished question, and a station left half-done is the thing to open next lesson. Do not mark the lesson complete for a pupil who did not finish it: the score screen already shows what was and was not done, and that is the honest record.

Exit, next lesson and homework

Exit check

One of these, written, not a show of hands:

- Writing a condition with a comparison operator
- Choosing between if, elif and else
- Joining conditions with and, or and not

The confidence columns on the worksheet are a supplement to this, not a substitute: a pupil who rates themselves confident and cannot answer the exit question has told you nothing.

Retrieval link to next lesson

Next is pr-104, Count-controlled loops. Use this lesson's big question as next lesson's starter, so the first thing they do is retrieve it.

Homework

Finish any station left open, on the website, from the question they stopped at. If the class has no devices at home, set the exam practice on the worksheet instead and mark it next lesson.

Shorter lessons, longer lessons and no internet

If this lesson takes more than an hour

It will. There are 55 activities across 6 stations, and a beginner does not finish 55 of them in twenty-nine minutes. Plan it as two or three sessions of an hour: stations 1 and 2 in the first, 3 and 4 in the second, 5, 6 and the final challenge in the third. Each session keeps the same starter-explain-model opening; the online block resumes where the class got to. Honest total: two to three hours.

A 40 to 45 minute lesson

Keep the starter, the explanation and the model at full length: cutting those is what makes the online work fail. Take the time out of the online block and the apply phase. Pupils stop where they are; their progress saves after every answer and reopens at their earliest unfinished question, so the next lesson starts there.

A shorter lesson still does not authorise skipping a question. The save-and-resume point is wherever the pupil got to; there is no way to mark a question complete without completing it, and no teacher bypass.

If the internet or the devices are unavailable

Teach from the slides and the worksheet. The slides carry the same teaching as the station walls, and the worksheet carries the key facts, the challenges and the exam practice. Set the online stations when the devices are back.

There is no offline version of the experience, and this plan does not contain one. Do not hand out a printed walkthrough of the stations as a substitute: the experiences are used on the website.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.